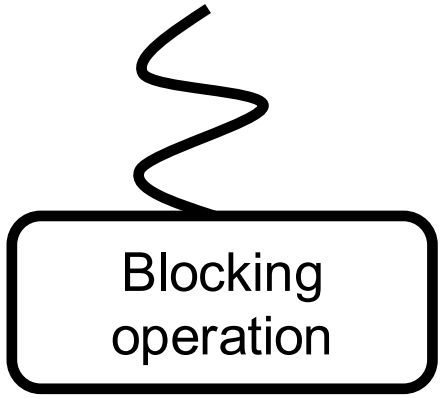
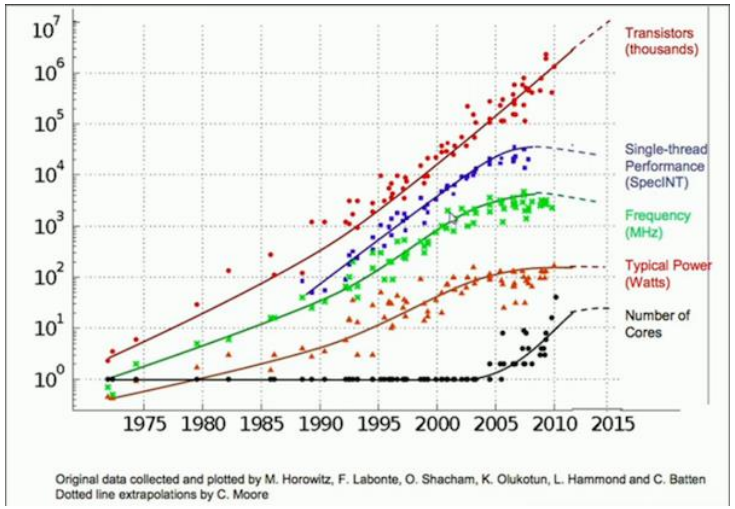
Concurrency



Thread





- A **critical section** is a piece of code that accesses a *shared* resource, usually a variable or data structure.
- A **race condition** (or **data race** [NM92]) arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.
- An **indeterminate** program consists of one or more race conditions; the output of the program varies from run to run, depending on which threads ran when. The outcome is thus not **deterministic**, something we usually expect from computer systems.
- To avoid these problems, threads should use some kind of **mutual exclusion** primitives; doing so guarantees that only a single thread ever enters a critical section, thus avoiding races, and resulting in deterministic program outputs.

Monitors Locks Semaphores
Condition Variables

Loads Stores Test&Set
Disable Interrupts

Review: Locks

Goal: Provide mutual exclusion (mutex)

Three common operations:

- Allocate and Initialize
  - `Pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;`
- Acquire
  - Acquire exclusion access to lock;
  - Wait if lock is not available (some other process in critical section)
  - Spin or block (relinquish CPU) while waiting
  - `Pthread_mutex_lock(&mylock);`
- Release
  - Release exclusive access to lock; let another process enter critical section
  - `Pthread_mutex_unlock(&mylock);`

Review: Locking Linked Lists

Can critical section be smaller?

Pthread_mutex_lock(&L->lock);

Finer-grained locking (still on entire list) permits more concurrency. However, must be careful.

Pthread_mutex_unlock(&L->lock);

~~Pthread_mutex_lock(&L->lock);~~

If no List_Delete(), locks not needed
(Any non-head element cannot change once inserted;
assume store to L->head is atomic)

~~Pthread_mutex_unlock(&L->lock);~~

```
Void List_Insert(list_t *L,  int key) {

  node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L, int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)

                return 1;
                tmp = tmp->next;
        }
    return 0;
}
```

Review: Lock Implementation Goals

Correctness

- Mutual exclusion
  - Only one thread in critical section at a time
- Progress (deadlock-free)
  - If several simultaneous requests, must allow one to proceed

Fairness

- Bounded (starvation-free)
  - Must eventually allow each waiting thread to enter
- Threads wait roughly same time when locking repeatedly

Performance

CPU is not used unnecessarily (e.g., spinning)

Review: LOCKS with Load+Store

Code uses a single **shared** lock variable

```
Boolean flag = false; // shared variable
Void lock(Boolean *flag) {
     while (*flag) /* wait */ ;
     *flag = true;
}
Void unlock(Boolean *flag) {
     *flag = false;
}
```



Use explicit hardware support!

Does this provide correct mutual exclusion?

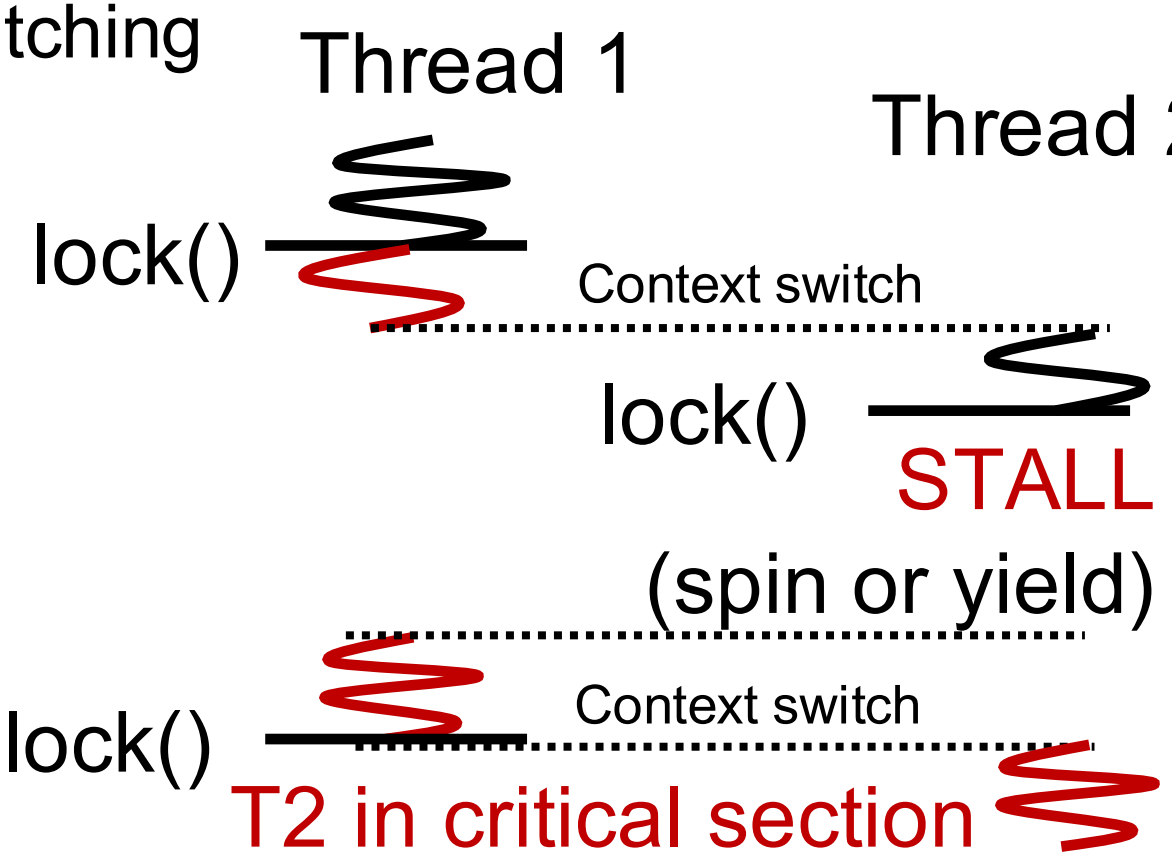
Can you produce an example schedule that fails with 2 threads?

Locking != No interruption

- In general, locks do not prevent context switching
- Locks prevent interleaving of critical section instructions across threads even under context switching

```
Void List_Insert(list_t *L,  int key) {
    node_t *new = malloc(sizeof(node_t));
    assert(new);
    new->key = key;
    lock(&l);
    new->next = L->head;
    L->head = new;
    unlock(&l);
}
```





Implementing Locks with Interrupts

Turn off interrupts for critical sections

- Prevent dispatcher from running another thread
- Code between interrupts executes atomically

```
void lock(lock_t *l) {
      disableInterrupts();
}

void unlock(lock_t *l) {
      enableInterrupts();
}
```

Disadvantages?

A useful idea for small critical sections, especially within OS … but not ideal for user-space

Don't want to allow user-space code disable interrupts

What if process crashes or runs away with the processor without yielding?

Stopping interrupts for too long is bad (can lose data)

Only works on uniprocessors

Implementing Locks with Atomics

Use hardware support for atomic operations

Atomic operation: No other instructions can interleave

Examples of atomic operations

- Loads and stores of words (aligned, hardware-native load/store size)
  - Load r1, B
  - Store r1, A
- **Special hardware instructions**
  - **Test&Set**
  - **Compare&Swap**
  - **Fetch&Add**

xchg: atomic exchange, or test-and-set

```
// xchg(int *addr, int newval)
// return what was pointed to by addr
// at the same time, atomically store newval into addr

int xchg(int *addr, int newval) {
    int old = *addr;
    *addr = newval;
    return old;
}

static inline uint
xchg(volatile unsigned int *addr, unsigned int newval) {
    uint result;
    asm volatile("lock; xchgl %0, %1" :
                 "+m" (*addr), "=a" (result) :
                 "1" (newval) : "cc");
    return result;
}
```

LOCK Implementation with XCHG

```
typedef struct __lock_t {
      int flag;
} lock_t;

void init(lock_t *lock) {
      lock->flag = ??;
}

void lock(lock_t *lock) {
      ????;
      // hint: spin-wait (do nothing)
}

void unlock(lock_t *lock) {
      lock->flag = ??;
}
```

Use
int xchg(int *addr, int newval)

LOCK implementation with XCHG

```
typedef struct __lock_t {
      int flag;
} lock_t;

void init(lock_t *lock) {
      lock->flag = 0;
}

void acquire(lock_t *lock) {
      while(xchg(&lock->flag, 1) == 1) ;
      // spin-wait (do nothing)
}

void release(lock_t *lock) {
      lock->flag = 0; // no atomics here
}
```

More atomic HW: Compare and Swap

```
int CompareAndSwap(int *addr, int expected, int new) {
  int actual = *addr;
  if (actual == expected)
    *addr = new;
  return actual;
}

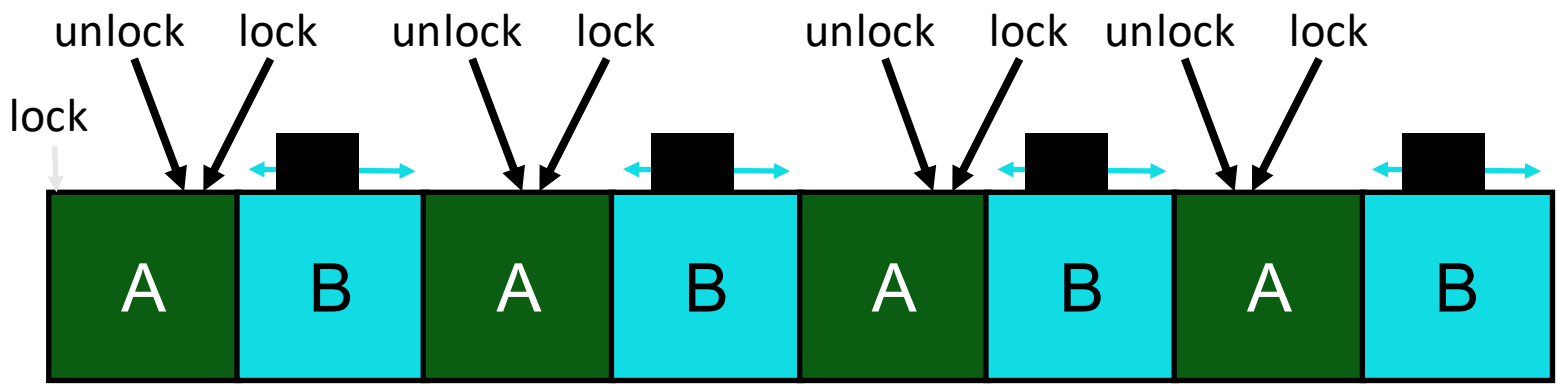
  void lock(lock_t *lock) {
      ??;
          // use CompareAndSwap(&lock->flag, ?, ?)
          // hint: spin-wait (do nothing)
  }
```

More atomic HW: Compare and Swap

```
int CompareAndSwap(int *addr, int expected, int new) {
  int actual = *addr;
  if (actual == expected)
    *addr = new;
  return actual;
}

 void lock(lock_t *lock) {
        while(CompareAndSwap(&lock->flag, 0, 1)
                                == 1) ;
        // spin-wait (do nothing)
 }
```

Basic Spinlocks can starve; unfair



OS Scheduler is typically independent of locks/unlocks

Fairness: Ticket Locks

Idea: reserve each thread's turn to use a lock.

Each thread spins until their turn.

Use yet another hardware atomic primitive, fetch-and-add:

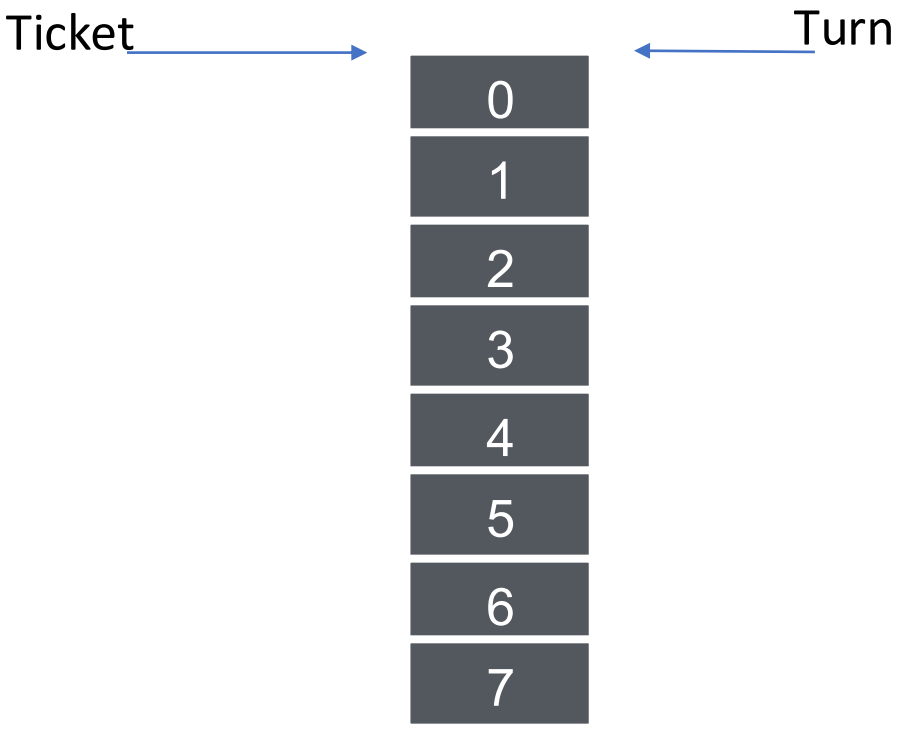
```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

Lock: Grab ticket; Spin while not thread's ticket != turn

Unlock: Advance to next turn

Ticket Lock: What we want

A lock():
B lock():
C lock():
A unlock():
B runs
A lock():
B unlock():
C runs
C unlock():
A runs
A unlock():
C lock():



Ticket Lock Example

A lock(): gets ticket 0, spins until turn = 0 →runs
B lock(): gets ticket 1, spins until turn=1
C lock(): gets ticket 2, spins until turn=2
A unlock(): turn++ (turn = 1)
B runs
A lock(): gets ticket 3, spins until turn=3
B unlock(): turn++ (turn = 2)
C runs
C unlock(): turn++ (turn = 3)
A runs
A unlock(): turn++ (turn = 4)
C lock(): gets ticket 4, runs



Ticket Lock Implementation

```
typedef struct __lock_t {
        int ticket;
        int turn;
}

void lock_init(lock_t *lock) {
        lock->ticket = 0;
        lock->turn = 0;
}
```

```
void lock(lock_t *lock) {
        int myturn = FAA(&lock->ticket);
        while (lock->turn != myturn); // spin
}

void unlock (lock_t *lock) {
        FAA(&lock->turn);
}
```

Spinlock Performance

Fast when…

- many CPUs
- locks held a short time
- advantage: avoid context switch

Slow when…

- one CPU
- locks held a long time
- disadvantage: spinning is wasteful